

Phase 2 — Data Preprocessing

Scope

This document covers Requirement 6: a Python preprocessing script that ingests the anonymized HCMUT FCSE dataset, imputes missing fields, synthesizes invigilator location preferences, and **instantiates** all the sets and parameters defined abstractly in Phase 1. The output is a single structured JSON file consumed by the Phase 3 solver.

Overview

The preprocessing pipeline performs eight steps:

1. Load raw CSV and impute missing values for `Thứ` and `Cơ sở`.
2. Construct the shift set E with all per-shift parameters.
3. Construct the invigilator set S and synthesize each invigilator's home campus $h(j)$.
4. Build the day-indexed family $\{E_d\}_{d \in D}$.
5. Build the back-to-back family E_b from same-day consecutive slot pairs.
6. Compute the sparse travel parameter d_{ij} .
7. Compute the workload target $\bar{W}$.
8. Extract the baseline schedule for later comparison (Phase 4).

The deliverable is a `preprocess.py` script and a serialized `iap_data.json` data file. All values from this document are deterministically reproducible given `RNG_SEED = 42`.

2.1 Loading and Imputation

The raw dataset contains 769 rows. Two columns have missing values:

Column	NaN count	Imputation rule
Thứ (day-of-week)	28	Derive from <code>Ngày</code> via <code>pd.to_datetime(...).dt.weekday</code> , then map $\{0, \dots, 6\} \rightarrow \{\text{Thứ 2}, \dots, \text{Chủ Nhật}\}$
Cơ sở (campus)	28	Derive from <code>Nhiệm vụ[0]</code> : $L \rightarrow \text{Cơ sở 1}$, $D \rightarrow \text{Cơ sở 2}$

Why these rules are sound

Both columns are functionally redundant with information already present in other columns. `Thứ` is fully determined by `Ngày` (a calendar date). `Cơ sở` is fully determined by the first letter of `Nhiệm vụ`, because the task codes embed the campus identifier (`LTK_*` = Lý Thường Kiệt = CS1, `DiAn_*` = Dĩ An = CS2). The imputation introduces no information that isn't already in the dataset — it just recovers it.

After imputation, all 769 rows are complete (`df.isna().sum().sum() == 0`).

2.2 Constructing the Shift Set E

Each unique value of `MS Ca thi` defines one shift. The shift identifier follows the format `YYYYMMDD_s` where the suffix $s \in \{1, \dots, 5\}$ encodes the time slot directly. For each shift i , we record:

Attribute	Source
P_i	row count grouped by <code>MS Ca thi</code> — all roles aggregated
$c(i)$	from <code>Cơ sở</code> (post-imputation)
$d(i)$	from <code>Ngày</code>
$s(i)$	parsed as <code>int(shift_id.split("_")[1])</code>
μ_i	computed via Eq. (1.1) from the slot and day multipliers

The five-slot timetable is verified to be exactly:

Slot s	Start time	μ_s^{slot}
1	07:00	1.1
2	09:30	1.0
3	13:00	1.0
4	15:30	1.0
5	18:15	1.2

All shifts have $\tau_i = 150$ minutes, confirming the simplification noted in Phase 1 §5 (3) — the duration factor cancels from the goal constraint, so $w_i = \mu_i$ in (5.3).

2.3 Invigilators and Synthetic Homes

The invigilator set S is the sorted set of unique values in MS của CÁN BỘ CỎI THI . For each $j \in S$ we draw $h(j)$ once from the categorical distribution given in Eq. (1.2):

```
rng = random.Random(RNG_SEED)
for cb in sorted_invigilators:
    r = rng.random()
    if r < 0.40:      homes[cb] = "CS1"
    elif r < 0.80:    homes[cb] = "CS2"
    else:             homes[cb] = "eq"
```

Because the RNG is seeded and invigilators are sorted before iteration, the same realization is produced on every run and on every machine.

2.4 Day Index $\{E_d\}$ and Back-to-Back Pairs E_b

The day index is built directly:

$$E_d = \{i \in E : d(i) = d\} \quad \forall d \in D \quad (2.1)$$

The back-to-back family is constructed from a fixed template of qualifying slot pairs:

$$\mathcal{B} = \{(1, 2), (3, 4), (4, 5)\} \quad (2.2)$$

Then for each date $d \in D$ and each pair $(s_1, s_2) \in \mathcal{B}$, if shifts exist at both slots on d , the pair of shift identifiers is added to E_b .

Why (2, 3) is excluded from $\mathcal{B}$

Slot 2 ends at 12:00 and slot 3 starts at 13:00 — a one-hour gap, sufficient for a meal break. By contrast, slot 4 ends at 18:00 and slot 5 starts at 18:15 (15-minute transition), and slots 1→2, 3→4 are immediately contiguous. These three pairs are the genuine fatigue concerns.

2.5 Travel Parameter d_{ij}

The travel parameter is materialized in a **sparse** dictionary keyed first by shift and then by invigilator, storing only nonzero entries. The construction follows Eq. (1.3) directly:

```
for sid, s in shifts.items():
    for cb in invigilators:
        if homes[cb] == "eq":          cost = 0.5
        elif homes[cb] == s["campus"]: cost = 0.0
        else:                          cost = 1.0
        if cost > 0:
            d_ij[sid][cb] = cost
```

Storing only nonzeros has two effects: it cuts the JSON footprint roughly in half, and it lets the Phase 3 solver iterate only over assignments that actually contribute to Z_{travel} .

2.6 Workload Target $\bar{W}$

Computed once via Eq. (1.4):

$$\bar{W} = \frac{1}{|S|} \sum_{i \in E} \mu_i \cdot P_i$$

The numerator is the *total weighted demand* across all shifts; dividing by $|S|$ gives each invigilator's fair-share target.

2.7 Baseline Schedule Extraction

The dataset already contains the manually-produced assignment as the MS của CÁN BỘ COI THI column. The script extracts this into a `baseline` dictionary mapping each shift identifier to its list of assigned invigilators. This serves two purposes:

1. **Validation:** the baseline must be feasible under hard constraints (H1) and (H2); if not, the constraints are mis-specified.
 2. **Comparison anchor:** Phase 4 compares the ILP solution against this baseline on travel cost, fatigue events, and workload deviation.
-

Realized Problem Instance

Running `preprocess.py` on the HCMUT dataset yields:

Cardinalities

Set	Symbol	Value
Exam shifts	$\ E\ $	65
Invigilators	$\ S\ $	73
Distinct dates	$\ D\ $	23
Campuses	$\ C\ $	2
Back-to-back pairs	$\ E_b\ $	24

Derived values

Quantity	Value
Total invigilator-shift demand $\sum_i P_i$	769
Total weighted demand $\sum_i \mu_i P_i$	≈ 797.74
Workload target $\bar{W}$	≈ 10.928
Nonzero d_{ij} entries	2,822 of 4,745 ($\approx 59\%$)

Realized synthetic home distribution

With `RNG_SEED = 42` and $|S| = 73$:

Home	Count	Share
CS1 (LTK)	35	48.0%
CS2 (DiAn)	24	32.9%
eq	14	19.1%

The realized split deviates from the target 40/40/20 due to sample variance at $n = 73$ — this is expected and does not warrant resampling, since a fixed seed is more important for reproducibility than matching the target distribution exactly.

Problem size (instantiated)

Component	Count
x_{ij} binary variables (dense)	$\leq 4,745$

Component	Count
y_{iij} binary variables (dense)	$\leq 1,752$
α_j^-, α_j^+ continuous variables	146 total
(H1) coverage constraints	65
(H2) daily-cap constraints (dense bound)	$\leq 1,679$
(S2) fatigue link constraints	$\leq 1,752$
(S3) workload goal constraints	73
Total variables (dense)	$\sim 6,500$
Total constraints (dense)	$\sim 3,500$

This problem size is comfortably within the operating range of any open-source ILP solver. Expected solve time: well under one second on a modern laptop.

Data Observations

A few useful patterns surface during preprocessing — these establish the *motivation* for the optimization (what's wrong with the baseline) without prejudging the Phase 4 analysis.

Baseline workload imbalance

The manual baseline assigns invigilators between **3 and 19 shifts** each, with mean ≈ 10.5 and standard deviation ≈ 2.2 . The most-loaded invigilator does roughly six times the work of the least-loaded one. This is the unfairness our L_1 workload goal aims to compress.

Travel cost is non-trivial

Of the 4,745 feasible (shift, invigilator) pairs, 59% involve some travel cost ($d_{ij} > 0$). The λ_1 weight will therefore pull strongly on the optimum; tuning will need to balance travel cost against fairness deviation rather than treating either as dominant.

Multi-shift days are common in the baseline

A total of 226 (invigilator, day) pairs have at least 2 shifts in the baseline, with the observed maximum being 4. This justifies $k = 4$ as the daily cap in (H2): a tighter cap would make

even the baseline infeasible. The fatigue penalty (S2) is the right mechanism to discourage stacking *softly*, rather than prohibiting it.

Output Schema

The serialized JSON contains:

```
iap_data.json
├─ shifts          dict: shift_id → {slot, date, weekday, campus, P, mu}
├─ invigilators    list[str]
├─ homes           dict: cb_id → "CS1" | "CS2" | "eq"
├─ E_d             dict: date → list[shift_id]
├─ E_b             list[[shift_id_1, shift_id_2]]
├─ d_ij            dict: shift_id → {cb_id: cost}  (sparse, nonzeros only)
├─ W_bar           float
├─ baseline        dict: shift_id → list[cb_id]
└─ config          dict: slot_mult, day_mult, daily_cap, rng_seed, ...
```

The Phase 3 solver reads this file and instantiates the model exactly once — no further data-side computation is needed during the optimization itself.